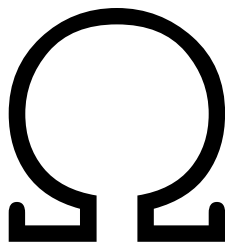


# Inverter Authorizer Update

## Audit Report

June 5, 2025



Team Omega

`teamomega.eth.limo`

|                                                                                                         |          |
|---------------------------------------------------------------------------------------------------------|----------|
| <b>Summary</b>                                                                                          | <b>2</b> |
| <b>Scope of the Audit</b>                                                                               | <b>3</b> |
| <b>Methods Used</b>                                                                                     | <b>3</b> |
| <b>Resolution</b>                                                                                       | <b>3</b> |
| <b>Disclaimer</b>                                                                                       | <b>3</b> |
| <b>Severity definitions</b>                                                                             | <b>4</b> |
| <b>Findings</b>                                                                                         | <b>4</b> |
| AUT_Roles_v1.sol                                                                                        | 4        |
| AR1. Unnecessary check in burnAdminRole [info] [resolved]                                               | 4        |
| AR2. GAP variable non updated [info] [resolved].                                                        | 4        |
| AR3. Unbounded loop in removeAccessPermission [info] [not resolved].                                    | 5        |
| AUT_EXT_VotingRoles_v1.sol                                                                              | 5        |
| AEVR1. Motions use threshold value from when created but with current voter set [medium] [not resolved] | 5        |
| AEVR2. addVoter does not validate the threshold [low] [resolved]                                        | 6        |

## Summary

Inverter has asked Team Omega to audit an update of the Authorization logic in their contracts.

We found **no high severity issues** - these are issues that can lead to a loss of funds, and are essential to fix. We classified **no** issues as “medium” - this is an issue we believe you should definitely address. In addition, **2** issues were classified as “low”, and **1** issue was classified as “info” - we believe the code would improve if these issues were addressed as well.

| Severity | Number of issues | Number of resolved issues |
|----------|------------------|---------------------------|
| High     | 0                | 0                         |
| Medium   | 1                | 0                         |
| Low      | 1                | 1                         |
| Info     | 3                | 2                         |

## Scope of the Audit

The audit concerns changes in the Solidity files in the following PR:

<https://github.com/InverterNetwork/contracts/pull/752>

Or, more precisely, the differences between `33c7565ad7bf01ab1aea9cf5b9ebe916fe4d6f23` (the base commit) and `da8f95b37303ae32f24ca127e883b46fec2ed874` (the updated commit).

These changes regard the authorization logic, which was completely refactored into a generic mechanism that governs permissions over the entire family of Inverter contracts.

## Methods Used

### Code Review

We manually inspected the source code to identify potential security flaws.

The contracts were compiled, deployed, and tested in a test environment.

### Automatic analysis

We have used static analysis tools to detect common potential vulnerabilities. The tools have detected a number of low severity issues, concerning mostly the variables naming and external calls, were found.

We have included any relevant issues below in the appropriate parts of the report.

## Resolution

The issues were addressed on commit `33d92b945fcf97b0bf2a272b3c4a6d13a026ff35`

We audited the fixes and updated the issues below.

## Disclaimer

The audit makes no statements or warranties about utility of the code, safety of the code, suitability of the business model, regulatory regime for the business model, or any other statements about fitness of

the contracts to purpose, or their bug free status. The audit documentation is for discussion purposes only.

## Severity definitions

|               |                                                                                                                               |
|---------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>High</b>   | Vulnerabilities that can lead to loss of assets or data manipulations.                                                        |
| <b>Medium</b> | Vulnerabilities that are essential to fix, but that do not lead to assets loss or data manipulations                          |
| <b>Low</b>    | Issues that do not represent direct exploit, such as poor implementations, deviations from best practice, high gas costs, etc |
| <b>Info</b>   | Matters of opinion                                                                                                            |

## Findings

### AUT\_Roles\_v1.sol

#### AR1. Unnecessary check in burnAdminRole [info] [resolved]

The `burnAdminRole` function has a check that if `getRoleAdmin(roleId_) == BURN_ADMIN_ROLE` it will just return. However, since the function is protected with the modifier of `onlyRole(getRoleAdmin(roleId_))`, and the `BURN_ADMIN_ROLE` is assigned to manage itself on `init`, this check can never be true and can be removed.

*Recommendation:* Remove the check from lines 491 to 493.

*Severity:* Info

*Resolution:* The issue was resolved as recommended

#### AR2. GAP variable non updated [info] [resolved].

A few new state variables were added, but the `gap` remains of the same size This is fine only if the contract is not already deployed or if it is only used for new deployments. But if the goal is to upgrade existing contracts this is not correct use of the `gap`, and risks making wrong storage calls for contracts inheriting this one.

*Recommendation:* Change the size of the `gap` variable

*Severity:* Info

*Resolution:* The team indicated that the intention is to do a fresh deployment of the contracts, and so the issue does not apply.

### AR3. Unbounded loop in removeAccessPermission [info] [not resolved].

In `removeAccessPermission` the code loops over the array of permissions to find the permission to be removed. If the list of permissions is very long, and the permission to remove is towards the end of the list, the function can run out of gas, making it very hard to remove that permission. This scenario is not very likely to occur, as it is not very likely that so many permissions will be used.

*Recommendation:* Add an `index` to the function arguments to find the permission to remove in constant time.

*Severity:* Info

*Resolution:* The team indicated that the list of permissions that will be defined will be of a size that makes the current issue not applicable.

## AUT\_EXT\_VotingRoles\_v1.sol

### AEVR1. Motions use threshold value from when created but with current voter set [medium] [not resolved]

When a motion is created, it stores the `requiredThreshold` value as the threshold value at the time the motion was created. If the set of voters and the threshold are changed during the time the motion is ongoing, the new set of voters together with the threshold value stored with the motion are used to determine the outcome of the motion. This could make the required threshold too high or too low, risking making the motion impossible to pass (if for example the threshold is higher than the number of voters), or easier to pass than the minimum expected threshold of voters (if, say, the intention is to require a majority vote).

*Recommendation:* One possible approach is to save the original voter set when the motion was created and use that instead of the current set. Alternatively, you save the accounts that voted with the motion - but you should take care to remove those votes if a voter is removed from the voter set.

*Severity:* Medium

*Resolution:* The team acknowledged the issue but decided not to address it directly in the code. Instead, clear communication to the users should mitigate the issue, and the issue can be addressed in a future iteration of the code.

## AEVR2. addVoter does not validate the threshold [low] [resolved]

The `addVoter` function does not validate the threshold after adding the voter. This means the threshold might be invalid after the call to `addVoter`. For example, in the case where there are 2 voters and the threshold is set to 1, after `addVoter` there will be 3 voters, but the threshold will still be 1, which according to `validateThreshold` is not a valid configuration.

*Recommendation:* Verify the threshold after adding the voter in `addVoter`. You could also allow updating the threshold from the same call to make sure the operation can be done in a single function call.

*Severity:* Low

*Resolution:* The issue was resolved as recommended.